

Software & Machine Learning Taskphase Report

Conceptual Report — Tasks 1 to 5

Atharva Nischal

Reg. No.: 24961214

B.Tech – Cyber Physical Systems, MIT Manipal

Domain: Software / Machine Learning

Repository: https://github.com/jarvixx28/Synergy_TP

Submission Date: 28 June 2026

Abstract

This report covers the theoretical concepts and design decisions underlying Tasks 1 to 5 of the Software/ML Taskphase. Topics include Git-based version control, Python virtual environments, Linux command-line basics, Python file handling, manual CSV parsing, pandas-based data analysis, systematic data cleaning, dataset validation, and data visualization using Matplotlib. The report focuses on the reasoning behind implementation choices rather than code details. It traces the full data pipeline — from raw CSV input through structured cleaning and validation to graphical output — and reflects on the software engineering principles reinforced at each stage. Word count: approximately 130 words.

1. Introduction

The Software/ML Taskphase is a sequenced series of assignments designed to build practical competence in software engineering and data processing. Tasks progressed from repository setup and environment management through Python fundamentals, CSV parsing, data cleaning, validation, and visualization. Each task relied on outputs from the previous one, creating a coherent end-to-end pipeline. The goal was not just to produce correct outputs but to understand why each technique is used and when it applies in real-world workflows.

Table 1: Summary of Tasks 1–5 and Key Concepts

Task	Objective	Key Concept	Deliverable
1	Repo & environment setup	Git, GitHub, venv, Linux basics	Repo, README, .gitignore
2	CSV analysis with Python	File I/O, dicts, JSON output	Summary JSON
3	Manual parser vs pandas	CSV structure, DataFrames, aggregation	Two summaries + comparison report

4	Messy data cleaning	Deduplication, type conversion, imputation, validation	Cleaned CSV + reports
5	Visualization	Matplotlib plots, axis labelling	Three PNG charts + summary

2. Development Environment and Version Control

2.1 Git and GitHub

Git is a distributed version control system that records project snapshots called commits. Instead of duplicate folder copies, each commit stores exactly what changed and when. GitHub hosts these repositories remotely, enabling backup, evaluation, and collaboration. Throughout the taskphase, a single repository named Synergy_TP was used with task-specific folders (task_1/ through task_6/) to keep each assignment self-contained. Descriptive commit messages — such as "Completed Task 4 cleaning pipeline" — make the project history readable and traceable.

2.2 Branches and Repository Hygiene

Assignments were completed sequentially on the main branch. A clean repository excludes auto-generated files: compiled bytecode (`__pycache__`/), virtual environment directories, editor metadata, and operating-system files. Committing these wastes storage and obscures meaningful changes.

2.3 Python Virtual Environments and requirements.txt

A virtual environment isolates project dependencies from the system Python installation, preventing version conflicts across projects. All required packages are listed in requirements.txt so any collaborator can reproduce the exact environment with a single install command. This isolation is particularly important in data workflows where different projects may need incompatible library versions.

2.4 .gitignore

The .gitignore file instructs Git to exclude files that should never be committed: the virtual environment folder, `__pycache__`/, *.pyc bytecode, temporary editor files, and OS metadata

like `.DS_Store`. Keeping these out of version control ensures the repository stays focused on source code and required assets.

3. Python Fundamentals and File Handling

3.1 Functions and Modular Design

Every task was split into single-responsibility functions rather than written as one large script. The cleaning task alone had separate functions for loading data, removing duplicates, standardizing domains, converting each column type, handling missing values, validating, and writing reports. This modularity makes each unit testable in isolation, reduces bug-hunting time, and allows logic to be reused across tasks.

3.2 Lists, Dictionaries, and Type Hints

Lists store ordered collections such as parsed rows or column names; dictionaries map keys to values and were used to represent each CSV row (column name → cell value), to normalise domain labels (e.g. "machine learning" → "ML"), and to convert textual numbers ("nine" → 9). Type hints such as `file_path: str` and `-> dict` document the intended interface of each function, improving readability and enabling static analysis without changing runtime behaviour.

3.3 File I/O and Context Managers

Python's `with` statement was used consistently for all file operations. It guarantees the file is closed — even if an exception occurs mid-read — preventing resource leaks. Multiple formats were used: CSV for tabular data, JSON for machine-readable summaries, Markdown for documentation, and PNG for plots.

3.4 Exception Handling

Defensive programming was applied where inputs could be malformed: command-line arguments were validated before use, malformed CSV rows were skipped without crashing, and type-conversion errors (e.g. non-numeric scores) were caught and recorded. Proper error handling prevents silent data corruption and produces informative messages when something goes wrong.

4. CSV Parsing and Pandas

4.1 The CSV Format

A CSV (Comma-Separated Values) file stores tabular data as plain text. The first line is the header row; each subsequent line is a record whose fields are separated by commas. CSV files are universally supported, lightweight, and human-readable, making them the default interchange format for datasets in data science and machine learning workflows.

4.2 Manual Parsing

Task 3 required reading the file with `open()`, splitting each line on commas, mapping field values to header names via dictionaries, converting score strings to integers, and handling empty or malformed rows explicitly. This approach reveals what high-level libraries do automatically: tokenization, type coercion, missing-field detection, and structured memory representation. Writing these steps manually builds intuition that is valuable when debugging library-based pipelines later.

4.3 Pandas and the DataFrame

Pandas represents tabular data as a `DataFrame` — an in-memory table with labelled rows and columns. A single function call loads the same CSV that required dozens of manual steps. Built-in methods handle grouping, aggregation, filtering, type conversion, and missing-value detection with concise, readable code. The same statistical summaries produced by the manual parser were reproduced in a fraction of the lines using `pandas`.

4.4 Comparison of Approaches

Aspect	Manual Parsing	Pandas
Code volume	High	Low
Type conversion	Explicit, manual	Built-in
Aggregation	Custom logic	<code>groupby</code> / <code>mean</code>
Scalability	Limited	Optimised for large data
Learning value	Reveals internals	Reflects industry practice

Both implementations produced identical summaries, confirming correctness through cross-verification — an important testing principle.

5. Data Cleaning and Validation

5.1 Why Cleaning Matters

The dataset for Task 4 was deliberately messy: duplicate rows, mixed capitalization in category fields, numeric values stored as words ("nine"), heights in metres and centimetres, attendance with percentage symbols, invalid values (attendance of -10 or 105 %), and missing entries. Running analysis on uncleaned data produces incorrect statistics. Data cleaning is consistently reported as the most time-consuming step in real machine learning projects, often accounting for the majority of total effort.

5.2 Cleaning Steps Applied

- Deduplication — identical rows were removed first so all subsequent statistics count each student exactly once.
- Domain standardisation — variant spellings ("ml", "MACHINE LEARNING", "web development") were mapped to canonical labels (ML, Web, Electronics, Mechanical) before any grouping.
- Numeric conversion — percentage symbols were stripped from attendance; textual numbers were converted to integers using a lookup dictionary; heights were normalised to centimetres; weights to kilograms.
- Invalid value handling — attendance values outside 0–100 were replaced with NaN rather than kept, preventing them from skewing statistics.
- Missing value imputation — median imputation was applied to numeric columns. The median is more robust than the mean when data contains outliers, which was the case here given the invalid attendance values.
- Submitted normalisation — variants ("Y", "Yes", "N") were standardised to "yes" / "no".

5.3 Validation

Cleaning alone does not prove correctness. A separate validation module verified post-cleaning constraints: no duplicate student IDs, attendance within $[0, 100]$, numeric columns fully numeric, domain values restricted to the four canonical labels, and no missing values in critical columns. Separating validation from transformation means it can be run independently and reused in future tasks that consume the same dataset.

Table 2: Common Data Issues and Cleaning Rules Applied

Issue	Rule Applied	Reason
Duplicate rows	Drop exact duplicates	Avoid double-counting
Mixed domain names	Map to canonical label	Consistent grouping
Attendance with '%'	Strip symbol, cast to float	Enable numeric ops
Invalid attendance	Convert to NaN	Prevent skewed stats
Missing numerics	Median imputation	Outlier-robust fill
Textual scores/hours	Word-to-int dictionary	Uniform numeric type
Mixed height units	Convert all to cm	Consistent measurement

6. Data Visualization using Matplotlib

6.1 Role of Visualization

Numerical summaries alone can obscure patterns. Visualization converts tabular values into graphical forms that humans process quickly — trends, outliers, and group differences become immediately apparent. In machine learning, visualization also forms the core of exploratory data analysis (EDA) before model development.

6.2 The Three Plots

Average Score by Domain (bar chart) — compares academic performance across the four domains at a glance. A bar chart is appropriate because the x-axis is categorical (domain names) and the y-axis is continuous (average score).

Attendance vs Score (scatter plot) — places each student as a point; x = attendance percentage, y = score. Scatter plots preserve individual observations, enabling detection of trends or clusters. A positive correlation, if present, would appear as an upward drift of points from left to right.

Submission Status Count (bar chart) — shows how many students submitted versus how many did not. Two discrete categories with a count metric make a bar chart the natural choice.

6.3 Visualization Best Practices Applied

- Descriptive titles on every figure so the chart is self-explanatory without surrounding text.
- Labelled axes with units where applicable — omitting labels forces the viewer to guess what values mean.
- `tight_layout()` called before saving to prevent label truncation at figure edges.
- `savefig()` used instead of `show()` so plots are written to disk reproducibly without requiring a display.
- No seaborn — matplotlib was used directly as required, keeping the dependency footprint minimal.

6.4 End-to-End Data Pipeline

Figure 1: Workflow from raw CSV data to visualized output



7. Conclusion

The taskphase built a complete software engineering and data processing workflow across five tasks. Version control with Git and GitHub established project structure and traceability from the start. Python fundamentals — modular functions, dictionaries, file I/O, and exception handling — formed the implementation backbone. Manual CSV parsing revealed how data is represented at a low level before pandas abstracted those operations into concise, scalable code. The data cleaning task demonstrated that raw datasets are rarely ready for analysis: deduplication, unit normalisation, type conversion, invalid-value handling, and median

imputation were all necessary before the data was usable. Validation after cleaning enforced quality constraints and provided confidence in downstream tasks. Finally, Matplotlib turned the cleaned data into interpretable bar charts and a scatter plot, completing the pipeline from raw input to visual insight. Together, these tasks reflect the core stages of any real-world data science or machine learning project.

8. References

Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>

pandas Development Team. pandas User Guide. <https://pandas.pydata.org/docs/>

Matplotlib Development Team. Matplotlib Documentation. <https://matplotlib.org/stable/>

Git Project. Git Reference Manual. <https://git-scm.com/doc>

GitHub, Inc. GitHub Docs. <https://docs.github.com/>

McKinney, W. Python for Data Analysis. 3rd ed. O'Reilly Media, 2022.

VanderPlas, J. Python Data Science Handbook. O'Reilly Media, 2016.